第 1 章　Java 开发入门

一、

1．对象

2．面向对象、跨平台性

3．javac

4．Java 虚拟机（或 JVM）

5．JRE

二、

1．错

2．错

3．错

4．对

5．对

三、

1．C

2．ABCD

3．D

4．ABD

5．D

四、

1．简答性、面向对象、安全性、跨平台性、支持多线程、分布性。

2. Java 程序运行时，必须经过编译和运行两个步骤。首先将后缀名为.java 的源文件进行编译，生成后缀名为.class 的字节码文件，然后 Java 虚拟机将字节码文件进行解释执行，并将结果显示出来。

五、

HelloWorld.java

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("这是第一个 Java 程序!");
    }
}
```

第 2 章　Java 编程基础

一、

1．class

2．单行注释、多行注释、文档注释

3．4

4．==

5．容器、0

二、

1．对

2．错

3．对

4．错

5．对

三、

1．AC

2．A

3．BD

4．C

5．C

四、

1.Java 语言的八种基本数据类型有：byte 字节型，占一个字节。short 短整型，占两个字节。

int 整型，占 4 个字节。long 长整型，占 8 个字节。float 单精度浮点型，占 4 个字节。double 双精度浮点型，占 8 个字节。char 字符型，占两个字节。boolean 型，表示逻辑值，有 true 和 false 两个值，分别占一个字节。

2. 在 switch 条件语句和循环语句中都可以使用 break 语句。当它出现在 switch 条件语句中时，作用是终止某个 case 并跳出 switch 结构。当它出现在循环语句中，作用是跳出循环语句，执行循环后面的代码；continue 语句用在循环语句中，它的作用是终止本次循环，执行下一次循环。

五、

1.

```
public class getSum {
    public static void main(String[] args) {
        int sum = 0;
        for (int i = 1; i < 100; i++) {
            if (i % 2 != 0)
                sum += i;
        }
        System.out.println(sum);
    }
}
```

2.

```
public class Test {
    public static void main(String[] args) {
        int i = 1;
        long sum = 1;
        do {
             sum *= i;
             i++;
        } while (i <= 5);
```

```
            System.out.println(sum);
    }
}
```

第 3 章　面向对象（上）

一、

1．封装、继承、多态

2．private、default、protected、public

3．static

4．private

5．参数类型 参数个数

6．this

二、

1．错

2．对

3．错

4．对

5．对

三、

1．C

2．B

3．D

4．B

5．C

四、

1．

面向对象的特点主要可以概括为封装性、继承性和多态性。

其中封装是面向对象核心思想","将对象的属性和行为封装起来","不需要让外界知道具体实现细节","这就是封装思想。

继承性主要描述的是类与类之间的关系","通过继承","可以在无需重新编写原有类的情况下","对原有类的功能进行扩展。

多态性指的是在程序中允许出现重名现象","它指在一个类中定义的属性和方法被其它类继承后","它们可以具有不同的数据类型或表现出不同的行为","这使得同一个属性和方法在不同的类中具有不同的语义。

2.

(1)构造方法名与类名相同

(2)在构造方法名的前面没有返回值类型的声明

(3)在构造方法中不能使用 return 语句返回一个值

五、

Example.java

```
class Resume {

    private String name;

    private String sex;

    private int age;

    public Resume(){

    }

    public Resume(String name,String sex,int age){

        this.name = name;

        this.sex = sex;

        this.age = age;

    }

    public String getName(){

        return name;

    }

    public String getSex(){
```

```
            return sex;
        }
        public int getAge(){
            return age;
        }
        public void introduce(){
            System.out.println("姓名："+this.getName()+"\n性别："+this.getSex()+"\n年龄："+this.getAge());
        }
    }
    public class Example{
        public static void main(String[] args){
            Resume re = new Resume("李四","男",20);
            re.introduce();
        }
    }
```

第 4 章　面向对象（下）

一、

1．继承

2．重写

3．final

4．implements

5．抽象类

6．super

二、

1．错

2．对

3．对

4．对

5．对

6．错

7．对

三、

1．B

2．D

3．C

4．D

5．C

四、

1．概念：在 Java 中","类的继承是指在一个现有类的基础上去构建一个新的类","构建出来的新类被称作子类","现有类被称作父类","子类会自动拥有父类所有可继承的属性和方法。

好处：继承性主要描述的是类与类之间的关系","通过继承","可以无需重新编写原有类的情况下","对原有类的功能进行使用和扩展。

2．

1、应用程序不必为每一个子类编写功能调用","只需要对抽象父类进行处理即可。大大提高程序的可复用性。

2、子类的功能可以被父类的方法或引用变量所调用","这叫向后兼容","可以提高可扩充性和可维护性。

3、使用多态可以解决项目中紧偶合的问题","提高程序的课扩展性.是 OCP 原则的一个具体的实现。

五、

Employee.java

```
abstract class Employee{
```

```
    private String name;                        //定义姓名name并私有化属性
    private int month;                          //定义生日月份month并私有化属性
    public Employee(){}                         //无参构造器
    public Employee(String name,int month){  //有参构造方法
        this.name = name;    //给属性name初始化赋值
        this.month = month;  //给属性month初始化赋值
    }
    //获取属性name的方法
    public String getName(){
        return name;   //返回name属性
    }
    //获取属性month的方法
    public int getMonth(){
        return month;  //返回month属性
    }
    //给属性name赋初始值
    public void setName(String name){
        this.name = name;  //本类中的属性name
    }
    //给属性month赋初始值
    public void setMonth(int month){
        this.month = month; //本类中的属性month
    }
    //创建一个方法getSalary()用来计算工资，参数month是月份，如果当月是员工生日，奖励100元
    public double getSalary(int month){
        double salary = 0;     //定义工资变量
```

```
        //判断当前月份是否是员      工的生日月份，如果是奖励 100 元
        if(this.month == month){
            salary = salary + 100;
            return salary;    //返回工资 salary
        }
    }
}
```

SalariedEmployee.java

```
class SalariedEmployee extends Employee{
    private double monthSalary;           //封装 monthSalary 属性
    public SalariedEmployee(){}           //无参构造方法
    //有参构造方法   参数  姓名 生日月份  月薪
    public SalariedEmployee(String name,int month,double monthSalary){
        super(name,month);                //调用父类有参构造方法
        this.monthSalary = monthSalary;   //为属性 monthSalary 初始化赋值
    }
    //获取 monthSalary 的值
        public double getMonthSalary(){
        return monthSalary;
    }
    //给 monthSalary 赋值
    public void setMonthSalary(double monthSalary){
        this.monthSalary = monthSalary;
    }
    //覆盖父类中的方法
    public double getSalary(int month){
        double salary = monthSalary+super.getSalary(month);   //定义工资变量
```

```
        return salary;
    }
}
```

HourlyEmployee.java

```
class HourlyEmployee extends Employee{
    private double hourlySalary; //定义属性 hourlySalary 每小时的工资
    private int hours; //定义属性 hours 每月工作的小时数
    public HourlyEmployee(){}    //无参构造方法
    //有参构造方法  参数 姓名 生日月份  每小时的工资 每月工作的小时数
    public  HourlyEmployee(String  name,int  month,double  hourlySalary,int
        hours){
        super(name,month);                  //调用父类有参构造方法
        this.hourlySalary = hourlySalary ; //为属性 hourlySalary 初始化赋值
        this.hours = hours;                 //为属性 hours 初始化赋值
    }
    public double getHourlySalary(){    //获取 hourlySalary 的值
        return hourlySalary;
    }
    public int getHours(){            //获取 hours 的值
        return hours;
    }
    //定义 set 方法设置 hourlySalary  hours 的值
    public void setHourlySalary(double hourlySalary){
        this.hourlySalary =hourlySalary;
    }
    public void setHourly(int hours){
        this.hours = hours;
```

```
    }

    //覆盖父类方法

    public double getSalary(int month){

        if(hours < 0){      //如果工作小时数小于 0  输出数据错误

        System.out.println("数据错误");

        return 0;

        }

        //小于 160 个小时的 按照每个月的工作小时数乘以每小时的工资

        else if(hours <= 160)

            return hourlySalary*hours+super.getSalary(month);

        //超出 160 个小时的小时数 按照 1.5 倍计算

        else return hourlySalary*160+hourlySalary*1.5*(hours-
        160)+super.getSalary(month);

    }

}
```

SalesEmployee.java

```
class SalesEmployee extends Employee{

    private double sales ; //定义销售额 sales

    private double rate; //定义提成率 rate

    public SalesEmployee(){}

    public SalesEmployee(String name,int month,double sales,double rate){

        super(name,month);

        this.sales = sales;

        this.rate = rate;

    }

    public double getSales(){

        return sales;
```

```
    }

    public double getRate(){

        return rate;

    }

    public void setSales(double sales){

        this.sales = sales;

    }

    public void setRate(double rate){

        this.rate = rate;

    }

    public double getSalary(int month){

        return this.getSales()*(1+this.getRate())+super.getSalary(month);

    }

}
```

BasePlusSalesEmployee.java

```
class BasePlusSalesEmployee extends SalesEmployee{

    private double baseSalary; //定义基础工资 baseSalary

    //无参构造方法

    public BasePlusSalesEmployee(){}

    //有参构造方法

    public BasePlusSalesEmployee(String name,int month,double sales,double
    rate,double baseSalary){

        super(name,month,sales,rate);

        this.baseSalary = baseSalary;

    }

    //get/set 方法对私有属性的调用和设置

    public double gatBaseSalary(){
```

```
        return baseSalary;
    }
    public void setBaseSalary(){
        this.baseSalary = baseSalary;
    }
    public double getSalary(int month){
        return baseSalary+super.getSalary(month);
    }
}
```

Test.java

```
//定义一个测试类
public class Test{
    public static void main(String[] args){
    //声明一个 Employee 类型的数组，并创建不同子类型的对象
    Employee[] employee = {new SalariedEmployee(“张三”,1,6000),new
    HourlyEmployee(“李 四”,2,50,180),new SalesEmployee(“王
    五”,3,6500,0.15),new BasePlusSalesEmployee(“赵六”,4,5000,0.15,2000)};
    //打印每个员工的工资
    for(int i = 0; i < employee.length ;i++)
        System.out.println(Math.round(employee[i].getSalary(10)));
    }
}
```

第 5 章　异常

一、

1．RuntimeException

2．运行时异常　编译时异常

3．throw

4．throws

5．Exception

二、

1．错

2．对

3．对

4．对

5．错

三、

1．C

2．A

3．C

4．C

5．A

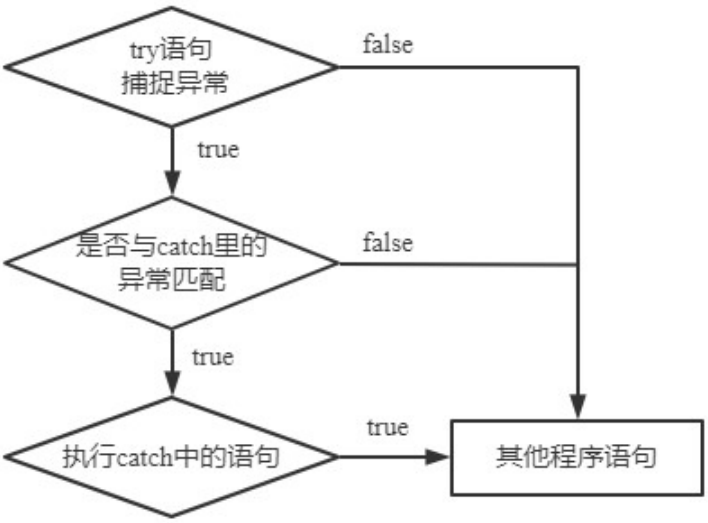
四、

1.

try、catch、finally、throw、throws。

2.

程序通过 try 语句捕获可能出现的异常，如果 try 语句没有捕获到异常，则直接跳出 try…catch 语句块执行其他程序；如果在 try 语句中捕获到了异常，则程序会自动跳转到 catch 语句中找到匹配的异常类型进行相应的处理。如果 try 语句捕获到的异常与 catch 语句例的异常匹配，则先执行 catch 中的语句，最后执行其他程序语句。



3.

处理编译时期的异常有两种方式如下：

（1）使用 try…catch 语句对异常进行捕获处理。

（2）使用 throws 关键字声明抛出异常，调用者对异常进行处理。

第 6 章　Java API

一、

1．String　StringBuffer　StringBuilder

2．length()

3．DateFormat

4．Random

5．edcba

二、

1．错

2．错

3．对

4．对

5．错

三、

1．B

2．C

3．A

4．B

5．A

四、

1．

String 类表示的字符串是常量，一旦创建后，内容和长度都是无法改变的。而 StringBuilder 和 StringBuffer 表示字符容器，其内容和长度可以随时修改。在操作字符串时，如果该字符串仅用于表示数据类型，则使用 String 类即可，但是如果需要对字符串中的字符进行增删操作，则使用 StringBuffer 与 StringBuilder 类。如果有大量字符串拼接操作，不要求线程安全的情况下，采用 StringBuilder 更高效。相反如果需要线程安全则需要使用 StringBuffer。

2．

基本数据类型	对应的包装类
byte	Byte
char	Character
int	Integer
short	Short
long	Long
float	Float
double	Double
boolean	Boolean

五、

1.

Example.java

```
import java.util.Random;

public class Example {
```

```
    public static void main(String[] args) {
        for(int i=0;i<10;i++){
            System.out.println(new Random().nextInt(100));
        }
    }
}
```

2.

Test.java

```
import java.text.DateFormat;
import java.util.Calendar;
import java.util.Date;
public class Test {
    public static void main(String[] args) {
        Calendar calendar = Calendar.getInstance();
        calendar.add(Calendar.DATE, 100);
        Date date = calendar.getTime();
        DateFormat format = DateFormat.getDateInstance(DateFormat.FULL);
        String string = format.format(date);
        System.out.println(string);
    }
}
```

第 7 章　集合类

一、

1．Collection

2．hashNext()　next()

3．Comparable

4．Key（键）、Value（值）

5．数组

二、

1．错

2．对

3．对

4．对

5．对

三、

1．D

2．D

3．C

4．A

5．C

四、

1．

List 的特点是元素有序、可重复。List 接口的主要实现类有 ArrayList 和 LinkedList。Set 的特点是元素无序、不可重复。Set 接口的主要实现类有 HashSet 和 TreeSet。Map 的特点是存储的元素是键(Key)、值(Value)映射关系，元素都是成对出现的。Map 接口的主要实现类有 HashMap 和 TreeMap。

2．

由于 ArrayList 集合的底层是使用一个数组来保存元素，在增加或删除指定位置的元素时，会导致创建新的数组，效率比较低，因此不适合做大量的增删操作。但这种数组的结构允许程序通过索引的方式来访问元素，因此使用 ArrayList 集合查找元素很便捷。

五、

1.

Example.java

```
1  public class Example {
2      public static void main(String[] args) {
3          ArrayList list = new ArrayList<>();
4          list.add("a");
5          list.add("b");
6          list.add("c");
7          list.add("a");
8          for(Iterator it = list.iterator();it.hasNext();){
9              System.out.println(it.next());
10         }
11     }
12 }
```

2.

Test.java

```
1  import java.util.*;
2  class Student {
3      private int age;
4      private String name;
5      public Student(int age, String name) {
6          this.age = age;
7          this.name = name;
8      }
9      public String toString() {
10         return age + ":" + name;
11     }
```

```
    public int hashCode() {
        return name.hashCode() + age;
    }
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (!(obj instanceof Student))
            return false;
        Student stu = (Student) obj;
        return this.name.equals(stu.name) && this.age == stu.age;
    }
}
public class Test {
    public static void main(String[] args) {
        HashSet<Student> hs = new HashSet<Student>();
        hs.add(new Student(18, "zhangsan"));
        hs.add(new Student(20, "lisa"));
        hs.add(new Student(20, "lisa"));
        System.out.println(hs);
    }
}
```

第 8 章　泛型

一、

1．参数化

2．泛型类　泛型接口　泛型方法

3．泛型方法

4．实现

二、

1．错

2．对

3．对

4．对

5．对

三、

1．D

2．A

3．A

4．AC

四、

1．

（1）提高类型的安全性。

泛型的主要目标是提高 Java 程序的类型安全性，即可以通过泛型定义对变量类型进行限制，在对变量类型限制之后编译器可以更好的进行类型验证。

（2）避免强制类型转换。

使用泛型的另一个好处是避免源代码中的许多强制类型转换。这使得代码可读性更高，并且减少出错机会。尽管减少强制类型转换可以降低使用泛型类代码的复杂度，但是声明泛型变量会带来相应的复杂操作。

五、

1.

```
interface Generic<T>{
public abstract void get(T t){}
}
class Generic<T> implements Generic{
 public void get(T t){}
```

```
}
```

第 9 章　反射

一、

1．动态

2．newInstance()

3．Method

4．getDeclaredFields()

5．Class

二、

1．对

2．对

3．对

4．错

5．对

三、

1．D

2．C

3．AC

4．A

5．C

四、

1．

Java 的反射（reflection）机制是指在程序的运行状态中，可以构造任意一个类的对象，可以得到任意一个对象所属的类的信息，可以调用任意一个类的成员变量和方法，可以获取任意一个对象的属性和方法。这种动态获取程序信息以及动态调用对象的功能称为 Java 语言的反射机制。

2．

（1）根据类名获取：类名.class；

（2）根据对象获取：对象.getClass()；

（3）根据全限定类名获取：Class.forName(“全限定类名”)。

第 10 章 I0（输入输出）

一、

1．字节流 字符流

2．Reader

3．BufferedInputStream　BufferedOutputStream

4．InputStreamReader　OutputStreamWriter

5．OutputStream

二、

1．对

2．错

3．错

4．对

5．错

三、

1．A

2．A

3．B

4．C

5．C

四、

1．

字节流的两个基类是 InputStream 和 OutputStream，字符流的两个基类是 Reader 和 Writer，它们都是 Object 类的直接子类，字节流是处理以 8 位字节为基本单位的字节流类；Reader 和 Writer 类是专门处理 16 位字节的字符流类。

2.

InputStreamReader 是 Reader 的子类，它可以将一个字节输入流转换成字符输入流，方便直接读取字符。OutputStreamWriter 是 Writer 的子类，它可以将一个字节输出流转换成字符输出流，方便直接写入字符。

五、

1.

Test01.Java

```
import java.io.*;
public class Test01 {
    public static void main(String[] args) throws Exception {
        // 字节流拷贝
        FileInputStream in = new FileInputStream("E:/src.txt");
        FileOutputStream out = new FileOutputStream("E:/des1.txt");
        byte[] buf = new byte[1024];
        int len;
        while ((len = in.read(buf)) != -1) {
            out.write(buf, 0, len);
        }
        in.close();
        out.close();
        // 字符流拷贝
        BufferedReader bf = new BufferedReader(new
                                                FileReader("E:/src.txt"));
        BufferedWriter bw = new BufferedWriter(new
                                                FileWriter("E:/des2.txt"));
```

```
        String str;
        while ((str = bf.readLine()) != null) {
            bw.write(str);
            bw.newLine();
        }
        bf.close();
        bw.close();
    }
}
```

第 11 章 JDBC

一、

1．java.sql.DriverManager

2．classpath

3．update

4．预编译

5．1

二、

1．错

2．对

3．对

4．对

5．错

三、

1．B

2．B

3．C

4．C

5．A

5．AD

四、

1．

1、加载并注册数据库驱动；

2、通过 DriverManager 获取数据库连接；

3、通过 Connection 对象获取 Statement 对象；

4、使用 Statement 执行 SQL 语句；

5、操作 ResultSet 结果集；

6、回收数据库资源。

2． 所谓预编译,就是说当相同的 SQL 语句再次执行时,数据库只需使用缓冲区中的数据,而不需要对 SQL 语句再次编译,从而有效提高数据的访问效率。

第 12 章 多线程

一、

1．Thread Runnable

2．新建状态（New）、就绪状态（Runnable）、运行状态（Running）

3．start()

4．sleep()

5．synchronized

二、

1．对

2．对

3．对

4．错

5．错

三、

1．C

2．B

3．B

4．D

5．BC

四、

1．

一种是继承 java.lang 包下的 Thread 类，覆写 Thread 类的 run()方法，在 run()方法中实现运行在线程上的代码。

```
new Thread() {

   public void run(){}

}.start();
```

另一种就是实现 java.lang.Runnable 接口，同样是在 run()方法中实现运行在线程上的代码。

```
class MyThread implements Runnable{

    public void run(){}

}
```

2．

同步代码块的作用是控制线程，保证同步代码块中只能有一个线程在运行，保证了多线程操作数据的安全性。

五、

1.

Example.java

```
public class Example {

    public static void main(String[] args) {
```

```
        TicketWindow tw = new TicketWindow();
        new Thread(tw, "线程1").start();
        new Thread(tw, "线程2").start();
    }
}
class TicketWindow implements Runnable {
    private int num = 100;
    public void run() {
        while (num > 0) {
            Thread th = Thread.currentThread();
            String th_name = th.getName();
            System.out.println(th_name + " 正在发售第 " + num-- + " 张票 ");
        }
    }
```

}

第 13 章 网络编程

一、

1. 链路层、网络层、运输层、应用层
2. UDP
3. TCP
4. DatagramSocket
5. ServerSocket

二、

1. 错
2. 错
3. 对

4．对

5．对

三、

1．D

2．A

3．D

4．C

5．D

四、

1．UDP 通信与 TCP 通信的区别在于，UDP 中只有发送端和接收端，不区分客户端与服务器端，计算机之间可以任意地发送数据；而 TCP 通信是严格区分客户端与服务器端的，在通信时，必须先由客户端去连接服务器端才能实现通信，服务器端不可以主动连接客户端，并且服务器端程序需要事先启动，等待客户端的连接。

2．在 TCP 连接中必须要明确客户端与服务器端，由客户端向服务器端发出连接请求，每次连接的创建都需要经过“三次握手”。第一次握手，客户端向服务器端发出连接请求，等待服务器确认；第二次握手，服务器端向客户端回送一个响应，通知客户端收到了连接请求；第三次握手，客户端再次向服务器端发送确认信息，确认连接。

五、

AskServer.java

```
/**
 *
 * 在线客服咨询人员
 *
*/
public class AskServer {
    public static void main(String[] args) {
        //创建 DatagramSocket，发送接收数据都依赖他
        DatagramSocket socket = null;
        try {
```

```
            socket = new DatagramSocket(8888);

            Scanner input = new Scanner(System.in);

            while (true) {

                //准备一个空的数据包，用来接收数据

               byte[] buf = new byte[1024];

               DatagramPacket packet = new DatagramPacket(buf, buf.length);

                //接收数据使用空的数据包

               socket.receive(packet);

                //输出信息

                String info = new String(packet.getData(), 0,
                packet.getLength());

                System.out.println("客户端请求：" + info);

                //判断是否退出

                if ("bye".equals(info)) {

                    break;

               }

                //发送数据

                String result = input.nextLine();

                byte[] buf2 = result.getBytes();

                DatagramPacket packet2 = new DatagramPacket(buf2,
                buf2.length,packet.getAddress(), packet.getPort());

                socket.send(packet2);

            }

        } catch (SocketException e) {

            e.printStackTrace();

         } catch (IOException e) {

            e.printStackTrace();
```

```
        } finally {
            //关闭 socket
            socket.close();
        }
    }
}
```

AskClient.java

```
/**
 *
 * 在线客服客户
 *
 */
public class AskClient {
    public static void main(String[] args) {
        //创建 DatagramSocket，发送接收数据都依赖他
        DatagramSocket socket = null;
        try {
            socket = new DatagramSocket(9999);
            Scanner input = new Scanner(System.in);
            while (true){
                //准备要发送的数据
                String str = input.nextLine();//bye
                //使用数据包把数据封装起来
                byte[] buf = str.getBytes();
                DatagramPacket packet = new DatagramPacket(buf, buf.length,
                      InetAddress.getByName("192.168.1.252"), 8888);
                //发送数据包
```

```
                socket.send(packet);
                //判断是否退出
                if ("bye".equals(str)) {
                    break;
                }
                //接收数据
                byte[] buf2 = new byte[1024];
                DatagramPacket packet2 = new DatagramPacket(buf2,
                buf2.length);
                socket.receive(packet2);
                System.out.println("服务器端反馈: "+
                new String(packet2.getData(), 0, packet2.getLength()));
            }
        } catch (SocketException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            //关闭 socket
            socket.close();
        }
    }
}
```